

1.First come first service

Aim: Program to implement first come first service scheduling algorithm

Algorithm

Step 1: Input the number of process required to be scheduled using FCFS.

Step 2: Input the arrival time and Burst time of each process

Step 3: Calculate Computation time by

If $top < at[i]$:

Top = $at[i] + bt[i]$

Step 4: Calculate **turnaround time** and **waiting_time**

Var = $ct[i] - at[i]$

tat.append[var]

var = $tat[i] - bt[i]$

wt.append(var).

Step 5: Caculate **average turnaround time** and waiting time

$a_tat = a_tat / n$

$a_wt = a_wt / n$

2.Shortest Job First

Aim: Program to implement shortest job first scheduling algorithm

Algorithm

Step 1:Input no of process required to be scheduling using SJF

Step 2: Sort all the process according to the arrival time.

For j in range (i+ 1,n):

If $l[i], l[j] = l[j], l[i]$:

$l[i], l[j] = l[j], l[i]$

Step 3: Select the process minimum arrival time and minimum burst time

Step 4: Calculate completion time for I in range(n);

$Ct = ct + l[i][1]$

Step 5:Calculate turn arrival time and waiting time ,

$TAT = tat + l[i][2]$

$$WT = \frac{1}{n} \sum_{i=1}^n [i] [2] - \frac{1}{n} \sum_{i=1}^n [i] [1]$$

Step 6: calculate average waiting time WT/n .

3.Round-Robin Scheduling

Aim : Program to implement Round robin scheduling algorithm

Algorithm

1.Enter the process and butstime

2.Enter the Quantum

3.if process burstntime is less than quantum no issue to execute but if greater than quantam it execute till quantum

then allocate to next process

while(tbt>ct):

 if(rt[i]>qt):

 rt[i] -=qt

 ct +=qt

 elif(rt[i]!=0):

 ct +=rt[i]

 rt[i]=0

 l[i].append(ct)

4.Calculate Waiting Time and Trun Around Time

5.Calculate average of Waiting time and Turn Around Time

print("Average Turn Around Time is:", tat/n)

print("Average Waiting Time is:", wt/n)

4. Priority Scheduling

Aim: Program to implement priority scheduling algorithm

Algorithm:

1. Enter the Process, Priority and burst time

2. Process execute in the basis of priority

`k=min(priorities)`

`priorities[priorities.index(k)]=max(priorities)+1`

3. Calculate waiting Time and Turn Around Time.

4. Calculate avg waiting time and Turn Around Time.

`print("Average Waiting time:" +str(sum(lis[0])/len(lis[0])))`

`print("Average turn around time:" +str(sum(lis[1])/len(lis[1])))`

5. Reader and writer problem using Semaphore

Aim :Program to implement reader/writer problem using semaphore

Algorithm

Step 1:Start the program .

Step 2:Import libraries for reader -Writer.

Import threading as thread

Step 3: Declare global variable which needs to be accessed while Reading as well as writing in a synchronous way

Global x

`x = 0`

`lock = thread.lock()`

Step 4: create a new function named Reader() which is used to read a global variable synchronously.

Def Reader ()

Global x

`lock.acquire()`

lock.release()

Step 5: Create new function named writer()

def Writer()

global x

lock.acquire()

x += 1

lock.release()

Step 6: Define main condition

If `__name__ == "__main__"`:

Step 7: Create two threads that try to access their reader and writer functions in a Synchronous way

Thread1.Start()

Thread2.Start()

6. Bankers Algorithm

Aim: Program to implement Bankers algorithm for deadlock avoidance

Step 1: start the program

Step 2: Let the total no of process be n and total number of resources be r m.

Step 3 : mention allocation resource as allocate, maximum resources that process need as max and available Resource as avail.

Step 4: determine need matrix for process by $need[i][j] = max[i][j] - alloc[i][j]$

Step 5 : consider the need matrix of every process and check whether any of them can be allocated the required resources from the available resources

if($need[i][j] > avail[i]$)

Step 6 : check if $need \leq available$ if so process granted and release resource to the system .

Step 7: It follows the Safety algorithm to check whether the system is in a safe state or not.

7.First in First out page replacement

Aim : program to implement first in first out page replacement

Algorithm

Step 1:Start to traverse the page

Step 2:import Queue from queue.

Step 3:define page faults function

```
Def pagefaults(incoming stream,n,frames)
```

Step 4: if current page is present on the memory do nothing

Step 5:Else pop the topmost page home the queue as it was inserted first

Step 6:Replace the topmost page with the current page from the string

if incomingStream[i] not ins:

```
val = queue.queue[0]
```

```
queue.get()
```

```
S.remove(val)
```

```
s.add(incomingStream[i])
```

```
queue.put(incomingStream[i])
```

```
page_faults += 1
```

Step 7: Increment the page faults

Step 8: stop.

8.Least recently used

Aim : Program to implement least recently used page replacement

Algorithm

Step 1:Start the program

Step 2:let the size =3 and refer the string

Step 3 : Initially all slots are empty so 7,0 ,1 are allocated to empty slots -> 3 page faults

Step 4: when 2 came it will take the place of 7 because it is least recently used -> 1 page faults

Step 5: 0 is already there so -> 0 page fault means page hit.

Step 6: When 3 came it will take place of 1 because it is least recently used -> 1 page fault

Step 7: process continues and gives the total number of page hits and page faults

Step 8: stop the program.

9. First fit for memory management

Aim: Program to implement first fit for memory management

Algorithm:

Step 1: Input memory blocks with size and processes with size.

Step 2: Initialize all memory blocks as free.

Step 3: Start by picking each process and check if it can be assigned to current block.

Step 4: If size-of-process $\leq$ size-of-block if yes then assign and check for next process.

For I in range (process)

For j in range (blocks)

if not occupied [j] and (block_size[j] $\geq$ Process_Size[i]):

allocate[i]=j

occupied[j]=True

Step 5: If not then keep checking the further blocks.

10. Best Fit for memory management

Aim: Program to implement best fit for memory management

Algorithm:

Step 1: Input memory blocks and processes with sizes.

Step 2: Initialize all memory blocks as free.

Step 3: Start by picking each process.

Step 4: find the minimum block size that can be assigned to

```
Bestidx = -1
```

```
if block_Size[j] >= process_size[i]:
```

```
if bestidx == -1:
```

```
bestidx = j
```

```
elseif block_Size[bestidx] > block_Size[i]:
```

```
bestidx = j
```

Step 5: If not then leave that process and keep checking

the further processes.

11. Worst fit for memory management

Aim : Program to implement worst fit for memory management

Algorithm:

Step 1: Input memory blocks and processes with sizes.

Step 2: Initialize all memory blocks as free.

Step 3: Start by picking each process .

Step 4: find the maximum block size that can be assigned to

```
Worstidx = -1
```

```
if block_Size[j] >= process_size[i]:
```

```
if worstidx == -1:
```

```
worstidx = j
```

```
elseif block_Size[worstidx] > block_Size[i]:
```

```
worstidx = j
```

Step 5: If not then leave that process and keep checking

the further processes.

12. Interprocess Communication

Aim: Program for interprocess communication

Algorithm:

Step 1: Start the program

Step 2: Import multiprocessing

Step 3: Input the values as numbers

Step 4: `def cal_square(numbers,q):`

`for n in numbers:`

`q.put(n*n)`

`if __name__ == "__main__":`

`numbers = [2, 4, 5]`

Step 6: Calculate to display Square , numbers

While `q.empty()` is False:

`Print(q.get())`